



Universitatea Tehnică "Gheorghe Asachi" din Iași
Facultatea de Automatică și Calculatoare
Domeniul Calculatoare și Tehnologia Informației
Specializarea Tehnologia Informației

Inteligență artificială

**Sistem de control și monitorizare a calității factorilor de mediu
folosind rețele bayesiene**

Studenti:

Avdei Elena, Grupa 1408A
Craiu Diana-Mihaela, Grupa 1408A

An universitar 2023-2024

Cuprins

Capitolul 1. Introducere.....	3
Capitolul 2. Descrierea problemei considerate.....	3
Capitolul 3. Structura aplicației.....	4
Capitolul 4. Interfață.....	5
Capitolul 5. Clasa BayesianNetwork.....	8
Capitolul 6. Teste automate pentru a demonstra buna funcționare a programului.....	8
6.1 Clasa pentru Testare unitară.....	8
6.2 Rezultate.....	8
Capitolul 7. Explicații suplimentare.....	9
7.1 Utilizarea inferenței prin enumerare în rețele bayesiene.....	9
Capitolul 8. Rolul fiecărui membru al echipei.....	9
Capitolul 9. Bibliografie.....	10

Capitolul 1. Introducere

Rețelele bayesiene, cunoscute și sub denumirile de rețele Bayes, rețele de credință sau rețele de decizii, constituie un cadru matematic și computațional avansat pentru modelarea relațiilor probabilistice între variabile.

Acestea oferă o perspectivă puternică asupra inferenței și procesului de luare a deciziilor în medii incerte. Central în structura lor este un graf aciclic și direcționat (DAG) care reprezintă variabilele și dependențele condiționale dintre acestea. Rețelele bayesiene reprezintă un instrument esențial în domenii diverse precum inteligența artificială și diagnosticarea medicală.

Bazându-se pe teoria probabilităților, aceste rețele permit modelarea eficientă a dependențelor dintre variabile și gestionează incertitudinea asociată cu procesul de decizie. Aceste calități le fac potrivite pentru diverse aplicații, inclusiv monitorizarea calității mediului, unde pot fi utilizate pentru a anticipa schimbările și a evalua riscurile asociate cu factorii de mediu. Prin actualizarea și ajustarea continuă a modelelor în funcție de noile informații, rețelele bayesiene oferă instrumente robuste pentru optimizarea procesului de luare a deciziilor în timp real.

O altă caracteristică semnificativă a rețelelor bayesiene este abilitatea lor de a gestiona incertitudinea în procesul de decizie. Teoria probabilităților pe care se bazează oferă un cadru formal pentru exprimarea și cuantificarea incertitudinii, permițând astfel reprezentarea și evaluarea riguroasă a riscurilor. Aceasta face rețelele bayesiene utile într-o gamă variată de domenii, de la analiza datelor până la sistemele de suport decizional.

Prin actualizarea constantă și ajustarea lor în funcție de noile informații disponibile, rețelele bayesiene demonstrează flexibilitate și adaptabilitate în fața schimbărilor în mediul lor. Această dinamicitate le conferă o importanță aparte în contextul evoluției continue a cunoașterii și tehnologiei, oferind un instrument de încredere pentru analiza și interpretarea datelor într-un mod actualizat și precis.

Capitolul 2. Descrierea problemei considerate

Problema fundamentală abordată în proiectul nostru constă în monitorizarea și evaluarea calității factorilor de mediu într-un mod precis și dinamic. De la calitatea aerului la calitatea apei și nivelul poluării, proiectul își propune să ofere o soluție cuprinzătoare pentru analiza și gestionarea impactului asupra mediului. Principala provocare constă în înțelegerea complexității interacțiunilor dintre diferiți factori de mediu și elaborarea unui sistem eficient care să permită luarea deciziilor informate.

Interfața dezvoltată pentru aplicație este implementată utilizând tehnologii Windows Forms în cadrul mediului de programare Visual Studio 2019. Aceasta oferă o interfață grafică intuitivă și prietenoasă pentru utilizator, facilitând interacțiunea acestuia cu sistemul. Utilizatorul are acces la diferite controale, cum ar fi ComboBox-uri, TextBox-uri și CheckBox-uri, pentru a introduce și a selecta date relevante, precum temperatură, viteza vântului, umiditate, activitate antropogenă, etc.

Interfața permite utilizatorului să introducă date specifice mediului înconjurător, să aleagă opțiunile relevante în funcție de activitatea umană și să primească estimări pentru diferiți factori. Poate, de asemenea, să exploreze diverse scenarii prin bifarea sau debifarea diferitelor opțiuni, cum ar fi influența traficului, a industriilor sau a altor factori. Această flexibilitate oferă o experiență interactivă și permite utilizatorului să obțină rapid informații relevante despre impactul unor factori specifici asupra calității mediului.

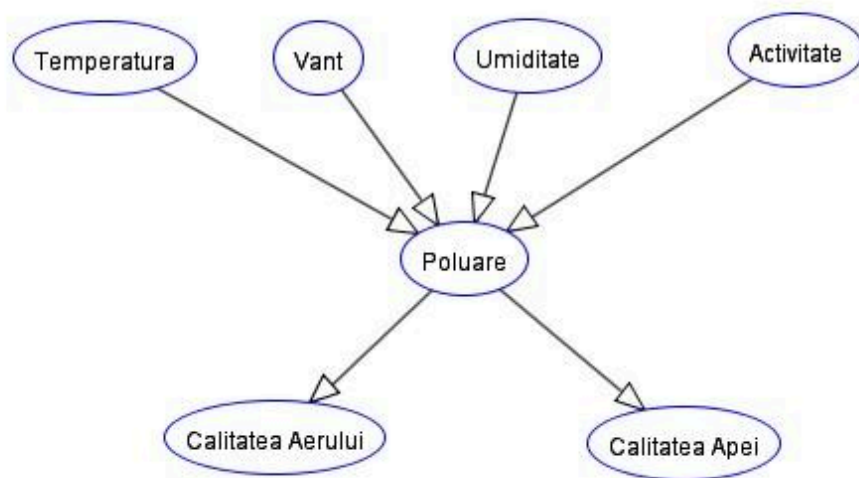


Fig.1 Arbore pentru rețeaua bayesiană bazată pe factori de mediu

Capitolul 3. Structura aplicației

Aplicația dezvoltată oferă utilizatorilor un program interactiv pentru analiza și predicția calității mediului înconjurător bazată pe o rețea bayesiană. Principalele clase sunt:

- **BayesianNetwork.cs**
Este clasa principală a proiectului care gestionează logica rețelelor bayesiene.
- **DataReader.cs**
Are rolul de a citi datele din fișierele de configurare și furniza informații relevante despre condițiile meteo și alte detalii specifice.

- **Details.cs**
Definește un obiect care conține informații detaliate despre o anumită valoare și probabilitățile asociate.
- **IDetailsFactory.cs**
Interfața IDetailsFactory definește un set de reguli pentru fabricile de detalii, furnizând o metodă GetDetails care trebuie implementată. Această metodă primește o valoare selectată și returnează un obiect de tipul Details, care conține informații detaliate despre acea valoare.
- **Link.cs**
Definește o legătură între două noduri într-o rețea Bayesiană. Această legătură este direcționată, având un nod părinte și un nod copil.
- **MarginalNodes.cs**
Gestionează nodurile marginale într-o rețea bayesiană, oferind metode pentru obținerea probabilității unui nod marginal specific și verificarea dacă o listă dată de noduri este exclusiv formată din noduri marginale.
- **MeteoDetails.cs**
Definește o serie de clase și interfețe. Fiecare clasă DetailsFactory este responsabilă pentru furnizarea detaliilor privind anumite aspecte ale mediului, cum ar fi temperatură, vânt, umiditate, activitate etc., bazându-se pe datele furnizate într-un fișier JSON. De asemenea, există clase specifice, precum TemperatureDetailsFactory, VantDetailsFactory, etc., care implementează interfața IDetailsFactory și furnizează detalii specifice pentru fiecare aspect al mediului menționat.
- **NivelPoluareMapper.cs**
Oferă funcționalități pentru maparea nivelurilor de poluare în funcție de valorile meteorologice specifice. Metodele acestei clase, cum ar fi MapTemperatura, MapUmiditate și MapVant, evaluează parametrii de intrare (temperatura, umiditatea, vântul) și returnează o etichetă corespunzătoare nivelului de poluare asociat. Etichetele posibile sunt "Scazut", "Moderat" și "Ridicat", în funcție de intervalul în care se încadrează valorile meteorologice.
- **Node.cs**
Reprezintă un nod în cadrul unei rețele bayesiene și conține informații despre variabilele și relațiile acestora.

Capitolul 4. Interfață

Form1

Ce doriti sa estimati:

- ☐ Temperatura
- ☐ Vant
- ☐ Umiditate
- ☐ Activitate antropogena
- ☐ Poluare
- ☐ Calitatea aerului
- ☐ Calitatea apei

Alegeti probabilitatea:

▼

Estimeaza

Introduceti datele cunoscute

Temperatura: °C

Umiditate: %

Vant: km/h

Trafic: ▼

Industrie: ▼

Poluare: ▼

Calitatea aerului: ▼

Calitatea apei: ▼

Rezultat

Nivelul estimat:

Probabilitatea:

Alte informatii:

▼

Despre

Iesire

Fig 2. Interfața grafică

Utilizatorul poate selecta din partea stângă ce nod vrea să interogheze și cu ce probabilitate și ce noduri participă la această interogare. De exemplu poate raspunde la întrebări de genul “Care este probabilitatea ca poluarea să fie moderată dacă temperatura este de 24 grade Celsius, umiditatea este de 56%, vântul are 12km/h , calitatea apei este scăzută și calitatea aerului este moderată?” Acele valori numerice se vor traduce în spate ca fiind una dintre valorile “Scăzut, Ridicat, Moderat” pentru a putea folosi rețeaua bayesiană. Am optat pentru această abordare pentru ca utilizatorul să poată introduce date cât mai exacte și să nu trebuiască să estimeze el nivelul acelor noduri.

Form1

Ce doriti sa estimati:

☐ Temperatura
☐ Vant
☐ Umiditate
☐ Activitate antropogena
☒ Poluare
☐ Calitatea aerului
☐ Calitatea apei

Alegeti probabilitatea:

Moderat ▼

Estimeaza

Introduceti datele cunoscute

Temperatura: 24 °C
Umiditate: 56 %
Vant: 12 km/h
Trafic: ▼
Industrie: ▼
Poluare: ▼
Calitatea aerului: Moderat ▼
Calitatea apei: Scazut ▼

Rezultat

Nivelul estimat: Moderat

Probabilitatea: 0.4998040768

Alte informatii:

Probabilitatea ca poluarea sa fie Moderat este de:
Moderat = 0.499804076801894

Despre

Iesire

Care este probabilitatea ca factorul 'Poluare' sa fie 'Moderat' atunci cand umatorii factori sunt:
temperatura: Moderat,
vant: Scazut,
umiditate: Moderat,
trafic: Nedeterminat,
industrie: Nedeterminat,
poluare: Nedeterminat,
calitate aer: Moderat,
calitate apa: Scazut?

Fig 3. Un caz de rulare

Capitolul 5. Clasa BayesianNetwork

Această clasă servește drept structură principală pentru construirea și gestionarea rețelelor bayesiene în cadrul proiectului, permițând analiza și evaluarea calității aerului pe baza diferitelor variabile. Este esențială pentru realizarea inferențelor probabilistice în contextul monitorizării calității aerului și a apei.

Metode:

1. **Sort(int[,] graph):** Implementează o sortare topologică pentru a asigura un ordin corect al evaluării în rețea.

```
// Calculate in-degrees for each node
int[] inDegrees = new int[vertices];
for (int i = 0; i < vertices; i++)
{
    for (int j = 0; j < vertices; j++)
    {
        if (graph[i, j] == 1)
        {
            inDegrees[j]++;
        }
    }
}

// Add nodes with no incoming edges to the set
for (int i = 0; i < vertices; i++)
{
    if (inDegrees[i] == 0)
    {
        noIncomingEdges.Add(i);
    }
}
```

```
while (noIncomingEdges.Count > 0)
{
    int node = noIncomingEdges.First(); // Remove a node from the set
    noIncomingEdges.Remove(node);
    sortedList.Add(node);

    for (int i = 0; i < vertices; i++)
    {
        if (graph[node, i] == 1)
        {
            // Remove the edge
            graph[node, i] = 0;
            inDegrees[i]--;

            if (inDegrees[i] == 0)
            {
                // If no more incoming edges, add to the set
                noIncomingEdges.Add(i);
            }
        }
    }
}

// Check for cycles
for (int i = 0; i < vertices; i++)
{
    if (inDegrees[i] > 0)
    {
        throw new InvalidOperationException("Graful are cel puțin un ciclu!");
    }
}
```


2. **EnumerationAsk(Node queryNode, Dictionary<Node, string> observedNodes):** Realizează inferență Bayesiană folosind metoda de enumerare.

```
0 references
public Dictionary<string, double> EnumerationAsk(Node queryNode, Dictionary<Node, string> observedNodes)
{
    //AddAllNodes();
    Console.WriteLine("Am intrat in EnumerationAsk");
    Dictionary<string, double> distribution = new Dictionary<string, double>();
    Dictionary<Node, string> obsNodes = observedNodes;
    string choice = observedNodes[queryNode];
    List<Node> nodes = new List<Node>();
    foreach(Node node in observedNodes.Keys)
    {
        nodes.Add(node);
    }
    foreach (var value in queryNode.PossibleValues)
    {
        observedNodes[queryNode] = value;
        double probability = EnumerateAll(*Nodes.ToList()*/nodes, obsNodes, observedNodes);
        distribution.Add(value, probability);
    }
    return Normalize(distribution);
}
```

3. **EnumerateAll(List<Node> variables, Dictionary<Node, string> obsNodes, Dictionary<Node, string> observedNodes):** Recursivitate pentru enumerarea tuturor posibilităților și calculul probabilităților.

```
3 references
private double EnumerateAll(List<Node> variables, Dictionary<Node, string> obsNodes, Dictionary<Node, string> observedNodes)
{
    Console.WriteLine("Am intrat in EnumerationAll");
    if (!variables.Any())
    {
        return 1.0;
    }

    var y = variables.First();
    variables.RemoveAt(0);

    if (observedNodes.ContainsKey(y))
    {
        var probability = CalculateConditionalProbabilityForNode(y, observedNodes, obsNodes);
        return probability * EnumerateAll(variables, obsNodes, observedNodes);
    }
    else
    {
        double sum = 0.0;
        foreach (var value in y.PossibleValues)
        {
            observedNodes[y] = value;
            var probability = CalculateConditionalProbabilityForNode(y, observedNodes, obsNodes);
            sum += probability * EnumerateAll(variables.ToList(), obsNodes, observedNodes);
        }
        return sum;
    }
}
```

4. **CalculateConditionalProbabilityForNode(Node node, Dictionary<Node, string> observedNodes, Dictionary<Node, string> nodes):** Calcularea probabilităților condiționate pentru fiecare nod dat abordând toate cazurile.

```

if (node.Name == "Temperatura" || node.Name == "Vant" || node.Name == "Umiditate" || node.Name == "Activitate")
{
    if (nodes[node] != "Nedeterminat")
        probability = (double)jsonData3[node.Name][nodes[node]];
    else
    {
        foreach (var value in node.PossibleValues)
            probability = (double)jsonData3[node.Name][value];
    }
    Console.WriteLine(probability);
    return probability;
}
Console.WriteLine(node.Name);
if (node.Name == "Apa" || node.Name == "Aer")
{
    Node parent = node.ParentNodes.First();
    Console.WriteLine($"Parintele e {parent.Name} cu valoarea {nodes[parent]}");
    if (nodes[parent] == "Nedeterminat")
    {
        foreach (var value in parent.PossibleValues)
        {
            if (nodes[node] != "Nedeterminat")
            {
                Console.WriteLine($"Cum se apeleaza jsonul: {node.Name} {value} {nodes[node]}");
                probability = (double)jsonData2[node.Name][value][nodes[node]];
            }
            else
            {

```

Această metodă folosește probabilități predefinite stocate în json-uri pentru a calcula probabilitățile condiționate.

Temperatura	Vant	Umiditate	Activitate	P(Poluare=S)	P(Poluare=M)	P(Poluare=R)
S	S	S	S	0.8	0.1	0.1
S	S	S	M	0.6	0.3	0.1
S	S	S	R	0.5	0.3	0.2
S	S	M	S	0.6	0.3	0.1
S	S	M	M	0.5	0.4	0.1
S	S	M	R	0.4	0.4	0.2
S	S	R	S	0.5	0.3	0.2
S	S	R	M	0.4	0.4	0.2
S	S	R	R	0.3	0.4	0.3
S	M	S	S	0.7	0.2	0.1
S	M	S	M	0.6	0.3	0.1
S	M	S	R	0.5	0.3	0.2
S	M	M	S	0.6	0.3	0.1
S	M	M	M	0.5	0.4	0.1
S	M	M	R	0.4	0.4	0.2
S	M	R	S	0.5	0.3	0.2
S	M	R	M	0.4	0.2	0.4
S	M	R	R	0.3	0.5	0.2
S	R	S	S	0.6	0.2	0.2
S	R	S	M	0.5	0.3	0.2
S	R	S	R	0.4	0.3	0.3
S	R	M	S	0.5	0.3	0.2
S	R	M	M	0.4	0.4	0.2
S	R	M	R	0.3	0.4	0.3
S	R	R	S	0.4	0.3	0.3
S	R	R	M	0.3	0.4	0.3
S	R	R	R	0.2	0.4	0.4
M	S	S	S	0.5	0.4	0.1
M	S	S	M	0.4	0.5	0.1
M	S	S	R	0.3	0.5	0.2
M	S	M	S	0.4	0.5	0.1
M	S	M	M	0.2	0.7	0.1
M	S	M	R	0.1	0.7	0.2
M	S	R	S	0.3	0.5	0.2
M	S	R	M	0.3	0.4	0.3
M	S	R	R	0.2	0.4	0.4
M	M	S	S	0.4	0.5	0.1
M	M	S	M	0.3	0.5	0.2

No observed value for this node.

OK Cancel

Probability Table for Aer			
Poluare	$P(Aer=S)$	$P(Aer=M)$	$P(Aer=R)$
S	0.0	0.2	0.8
M	0.1	0.8	0.1
R	0.8	0.2	0.0

No observed value for this node.

OK Cancel

Capitolul 6. Teste automate pentru a demonstra buna funcționare a programului.

6.1 Clasa pentru Testare unitară

- 1) **UnitTesting**: Această clasă conține un set de teste unitare pentru a verifica funcționalitatea și corectitudinea claselor proiectului. Fiecare test se concentrează pe o anumită funcționalitate și verifică rezultatele obținute comparativ cu rezultatele așteptate.

Descrierea rolului fiecărui test:

- ***EstimateTemperature_WhenAllMarginalNodes***: Verifică estimarea temperaturii atunci când toate nodurile completate în formular sunt marginale.
- ***EstimatePollution_WhenAllMarginalNodes***: Verifică estimarea poluării atunci când toate nodurile completate în formular sunt marginale.
- ***MapTemperatura_LowTemperature***: Verifică funcționarea corectă a metodei de mapare a temperaturii în clasa NivelPoluareMapper.
- ***MapUmiditate_MediumHumidity***: Verifică funcționarea corectă a metodei de mapare a umidității în clasa NivelPoluareMapper.
- ***MapVant_HighWind***: Verifică funcționarea corectă a metodei de mapare a vântului în clasa NivelPoluareMapper.

6.2 Rezultate

Test	Duration	Traits	Error Message
TestProject (5)	4.9 sec		
TestProject (5)	4.9 sec		
UnitTest1 (5)	4.9 sec		
EstimatePollution_Wh...	3.6 sec		
EstimateTemperature...	1.3 sec		
MapTemperatura_Lo...	< 1 ms		
MapUmiditate_Mediu...	< 1 ms		
MapVant_HighWind	< 1 ms		

Group Summary

TestProject

Tests in group: 5

Total Duration: 4.9 sec

Outcomes

5 Passed

Capitolul 7. Explicații suplimentare

7.1 Utilizarea inferenței prin enumerare în rețele bayesiene

Inferența prin enumerare este o metodă fundamentală în rețelele bayesiene utilizată pentru calcularea probabilităților condiționate sau a distribuțiilor de probabilitate într-o rețea bazată pe cunoștințe și dependențe probabilistice între variabilele aleatoare.

Această metodă este implementată în cadrul rețelelor bayesiene pentru a determina probabilitatea unui eveniment sau a unei variabile aleatoare date anumite informații sau evidențe referitoare la alte variabile din rețea.

Procesul de inferență prin enumerare implică:

1. **Specificarea evidenței:** Utilizatorul furnizează valorile cunoscute sau evidența referitoare la anumite variabile din rețea pentru a calcula probabilitatea altei variabile sau a unui eveniment.
2. **Calculul probabilităților condiționate:** Algoritmul efectuează calculele pentru a determina probabilitățile condiționate pe variabilele necunoscute, având în vedere evidența furnizată.
3. **Explorarea posibilităților:** Algoritmul explorează toate posibilitățile și combinațiile probabile ale valorilor necunoscute, aplicând regulile și dependențele din rețeaua bayesiană pentru a calcula probabilitățile dorite.
4. **Obținerea rezultatelor:** Rezultatul inferenței prin enumerare este o distribuție de probabilitate pentru variabilele de interes, care indică probabilitățile relative ale diferitelor valori ale acestor variabile, având în vedere evidența furnizată

Capitolul 8. Rolul fiecărui membru al echipei

- Avdei Elena:
 - Capitolele 2, 5, 7, 9 din documentație.
 - Clasa BayesianNetwork din cod.
 - Json-urile cu probabilitățile predefinite necesare rețelei bayesiene.
 - Clasele NivelPoluareMapper, Link, Node
- Craiu Diana-Mihaela:
 - Teste automate pentru a demonstra buna funcționare a programului.
 - Capitolele 1, 3, 4, 6 din documentație.
 - Interfața în Windows Form
 - Clasele DataReader, Details, MeteoDetails

Capitolul 9. Concluzie

Proiectul prezentat, bazat pe rețele bayesiene, oferă o abordare pentru monitorizarea și evaluarea calității mediului. Integrând sortarea topologică și metode de inferență Bayesiană, proiectul furnizează o aplicație pentru analiza dependențelor dintre variabilele de mediu. Această inițiativă nu doar optimizează procesul de luare a deciziilor, ci și subliniază importanța tehnologiei bayesiene în înțelegerea complexă a calității aerului și a apei.

Capitolul 10. Bibliografie

- https://pomegranate.readthedocs.io/_/downloads/en/stable/pdf
- Suport de curs Rețele bayesiene - Inteligență artificială -Florin Leon
- Curs 10. Rețele bayesiene - Inteligență artificială -Florin Leon